QueryGenie: A Self-Correcting Natural Language to SQL Interface

A report submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology
in
Computer Science & Information Technology

by

N V V S S Jayakanth Kamisetti	**2320090050**
Boddupally Narendar	**2320090009**
M Praneeth Reddy	**2320090018**

Under the Supervision of
M Parameswar



KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET

Hyderabad-500043, Telangana, India

2026

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is to certify that **N V V S S Jayakanth Kamisetti, Boddupally Narendar and M Praneeth Reddy** (ID.No. **2320090050, 2320090009 and 2320090018**), B.Tech (Computer Science & Information Technology), have worked on the project entitled **"QueryGenie: A Self-Correcting Natural Language to SQL Interface"** under my supervision and guidance. The content of this report is original and has not been submitted elsewhere for the award of any academic or professional degree.

.....

M Parameswar
Associate Professor,
Department of Computer Science & Information Technology,
Koneru Lakshmaiah Education Foundation, Telangana, India
Bowrampet, Hyderabad-500043

Date: .....

Acknowledgments

We thank our supervisor, M Parameswar, for the guidance given throughout this project, and the Department of Computer Science & Information Technology, Koneru Lakshmaiah Education Foundation, for the structure and review process that shaped it.

We are grateful to the authors of CodeS for releasing their code, their fine-tuned model weights and their evaluation setup openly; without them a faithful reproduction would not have been possible. We also thank the creators of the Spider benchmark and of its test-suite evaluator, and the open-source communities behind the tools this work builds on.

Abstract

Text-to-SQL models let people who do not know SQL query a database in plain English, but they answer every question with the same apparent confidence, so a wrong answer looks exactly like a right one. This project reproduces CodeS, an open-source text-to-SQL model (SIGMOD 2024), and extends it so that it says when it is unsure.

Using the released CodeS-1B checkpoint and the authors' prompt and inference procedure, we obtain 77.8% Execution Accuracy on the 1,034-question Spider development set with the official evaluator, against 77.9% in the paper. An ablation shows that every prompt component contributes 0.9 to 1.9 points.

The enhancement, confidence-aware abstention, executes the four candidate queries that beam search already produces and measures how many return the same result. When agreement is low the system declines and asks the user which reading was meant. At a pre-registered operating point it answers 90.4% of questions and is correct on 83.3% of them, against 77.8% when answering everything, and wrong answers fall from 230 to 156. The gain is significant (paired t-test $p = 7.7 \times 10^{-10}$; Wilcoxon $p = 9.8 \times 10^{-4}$; bootstrap 95% CI +4.5 to +6.7 points). On questions reworded without column names (Spider-Realistic), the reproduced model scores 69.5% against 70.9% in the paper; the wording costs 3.3 points, and abstention still removes a third of the wrong answers.

The system is delivered as QueryGenie, a desktop application that runs the evaluated pipeline locally on open weights.

Keywords: text-to-SQL, reproducibility, large language models, selective prediction, abstention.

Contents

Acknowledgments **ii**

List of Tables **v**

List of Figures **vii**

1 Introduction **1**

1.1 Problem and motivation 1

1.2 Objectives 2

1.3 Contributions and scope 2

2 Literature Review **4**

2.1 Research gap 5

3 Proposed Methodology **7**

3.1 Datasets 7

3.2 Architecture 8

3.3 The reproduced CodeS pipeline 8

3.4 Proposed approach: confidence-aware abstention 10

3.5 The application 12

4 Experimental Results **14**

4.1 Reproduction 14

4.2 Ablation study 15
4.3 Enhancement: confidence-aware abstention 15
4.4 Self-correction (application feature) 17
4.5 Benchmark-to-reality gap 17

5 Limitations and Future Work **19**

5.1 Limitations 19
5.2 Conclusion 20
5.3 Future work 20

References **22**

List of Tables

2.1 Related work most relevant to this project. 5

3.1 Evaluation sets. 8

4.1 Runs on the Spider development set (1,034 questions). "Simple" is the early metric; only "official" matches the paper. 15

4.2 Ablation study, official evaluator (runs AB-103 to AB-106). 15

4.3 Original against enhanced on the same 1,034 questions. 16

4.4 Significance over 10 calibration/test splits and a 5,000-sample bootstrap over questions. 16

4.5 The same 508 questions in their original and their realistic wording (run RL-001). 17

List of Figures

3.1 System architecture: reproduced CodeS components (blue), the enhancement (amber) and the application (green). 9
3.2 The CodeS pipeline, annotated from the paper. 10
3.3 QueryGenie, the desktop application. 13

4.1 Accuracy on answered questions against coverage; the dashed line is the original system (and random abstention). 16

Chapter 1

Introduction

1.1 Problem and motivation

Much of an organisation's data sits in relational databases, and getting answers out of them normally requires writing SQL. Staff who need a number — how many students failed more than two subjects, which department has the highest average marks — usually have to ask someone who can write the query. Text-to-SQL systems remove that step: they translate a plain-English question into a query, run it and return the result.

Recent language models do this well on benchmarks. The difficulty is what happens when they are wrong. A text-to-SQL system answers every question, and a wrong table looks just as authoritative as a correct one; for a user who cannot read SQL, silently wrong answers are the central risk. Most strong systems also depend on closed, hosted models, which means sending the database schema to a third party.

CodeS [1] addresses the second problem: its weights, code and evaluation setup are public, so it runs on local hardware and its numbers can be checked. It does not address the first. This project starts from the observation that CodeS already computes what is needed to know when it is unsure: it produces four candidate

queries with beam search and keeps one. If the candidates agree, the answer is more likely to be right; if they disagree, the question is probably ambiguous or hard. Using that agreement to decide when *not* to answer, and asking the user instead, turns a model that is sometimes silently wrong into one that says when it does not know.

1.2 Objectives

1. Reproduce the published Spider result of CodeS within the department's tolerance of ±5–10%, documenting every deviation from the authors' setup.
2. Establish which parts of the CodeS pipeline matter, through an ablation study.
3. Propose, implement and statistically test one enhancement: confidence-aware abstention.
4. Measure how the system holds up beyond the benchmark, on questions worded as users word them and on real-world databases.
5. Deliver the system as a working desktop application rather than a notebook.

1.3 Contributions and scope

The project contributes a faithful reproduction of fine-tuned CodeS-1B on Spider [2] (77.8% against 77.9%) with an audited evaluation harness; an ablation of the CodeS prompt; confidence-aware abstention with a pre-registered hypothesis and significance tests; a benchmark-to-reality evaluation on Spider-Realistic and BIRD; and QueryGenie, a desktop application for macOS and Windows.

CodeS is pre-trained on eight A800 80 GB GPUs, far beyond a student budget, so the project reproduces the published *inference-time* result with the released fine-tuned checkpoint, which is the configuration behind the 77.9% figure. Pre-training

and fine-tuning were not repeated.

Chapter 2

Literature Review

All papers reviewed were obtained from arXiv, as the department requires. Surveys of the field [3, 4, 5, 6, 7, 8] trace a shift from task-specific neural parsers to general language models prompted or fine-tuned for SQL. Table 2.1 summarises the fifteen works closest to this project.

Benchmarks and metrics. Spider [2] is the standard cross-domain benchmark; BIRD [18] moves to large real-world databases with dirty values, and Spider-Realistic [12], Spider-Syn [23] and Spider-DK [24] probe robustness to wording, synonyms and domain knowledge. Execution Accuracy has replaced exact string match as the preferred metric, because equivalent queries can differ in form.

Approaches. Graph-based encoders such as RAT-SQL [10] and LGESQL [25], table pre-training [26, 27] and constrained decoding [14] gave way to methods built on large language models [28, 19, 20, 21, 22, 29]. CodeS [1], reproduced here, instead builds open models by incrementally pre-training StarCoder [30]. Its reported Spider development-set results are 77.9% Execution Accuracy for CodeS-1B, 83.4% for 3B, 85.4% for 7B and 84.9% for 15B.

Table 2.1: Related work most relevant to this project.

Work	Year	Approach	Limitation for our purpose
Spider [2]	2018	Cross-domain benchmark; unseen databases at test time	Questions often name the columns; small, clean databases
Seq2SQL [9]	2017	Query generation with reinforcement learning	Single-table questions only
RAT-SQL [10]	2019	Relation-aware schema encoding and linking	Task-specific encoder; one answer, no confidence
BRIDGE [11]	2020	Adds matched cell values to the input	Value matching kept by CodeS; no uncertainty
Spider-Realistic [12]	2020	Spider questions without explicit column names	Evaluation set only
Test-suite accuracy [13]	2020	Executes queries on many database variants	Needs distilled databases
PICARD [14]	2021	Constrained decoding so only valid SQL is produced	Valid SQL can still be the wrong query
Self-consistency [15]	2022	Trusts the answer that most sampled reasoning paths reach	Not applied to SQL abstention
Selective classification [16]	2017	Declines low-confidence inputs; accuracy–coverage trade-off	Classification, not query generation
RESDSQL [17]	2023	Decouples schema linking from skeleton parsing	Same group as CodeS; answers everything
BIRD [18]	2023	Benchmark on large, real-world databases	Large tables, dirty values and domain knowledge; much harder than Spider
DIN-SQL [19]	2023	Decomposed prompting with self-correction from errors	Relies on a large closed model that reads error messages
DAIL-SQL [20]	2023	Systematic prompt engineering with LLMs	Closed models; schema sent to a third party
MAC-SQL [21], CHESS [22]	2023–24	Multi-agent pipelines and schema pruning for large databases	Several large-model calls per question
CodeS [1]	2024	Open 1B–15B models pre-trained on SQL-centric data	One query per question, no indication of reliability

2.1 Research gap

- **No reported confidence.** The reviewed systems answer every question with the same apparent confidence; none reports an accuracy–coverage trade-off or declines to answer.
- **Closed models.** Most of the strongest systems send the schema to a hosted model, which raises cost, privacy and reproducibility concerns.
- **The benchmark-to-reality gap.** Results are reported on Spider, whose questions often name the columns and whose databases are small; how a Spider-trained model behaves on realistic wording and real-world databases

is rarely measured for the same system.

This project uses the candidates CodeS already produces to measure agreement, abstains when agreement is low, evaluates the result as an accuracy–coverage trade-off with significance tests, and measures the same system beyond Spider.

Chapter 3

Proposed Methodology

3.1 Datasets

All data comes from Hugging Face, which is on the department's approved list; Kaggle was not used. Table 3.1 lists the three evaluation sets.

- **Spider** [2], the benchmark on which CodeS reports its results: questions, gold SQL and SQLite databases, with training and development sets on disjoint databases, so a model is always evaluated on schemas it has never seen. Questions from `xlangai/spider` (the authors' organisation), databases from `HAL-9001/spider-databases`.
- **Spider-Realistic** [12]: 508 Spider development questions rewritten by their authors so that they no longer name the columns ("from the oldest to the youngest" instead of "ordered by age"), with the same databases and gold SQL; from `aherntech/spider-realistic` (a copy of the authors' release, CC BY-SA 4.0).
- **BIRD Mini-Dev** [18]: the BIRD authors' 500-question subset over 11 real-world databases (finance, medicine, sport, schools and others) of up to 571 MB, with dirty values and an expert "evidence" hint per question; questions from

Table 3.1: Evaluation sets.

Set	Questions	Databases	Used for
Spider development	1,034	20	Reproduction, ablation, enhancement
Spider-Realistic	508	19	Realistic wording
BIRD Mini-Dev	500	11	Real-world databases

the official `birdsql/bird_mini_dev`, databases from `nlile/BIRD-bench`.

The Spider development databases are small: 2 to 11 tables (4.05 on average) and 7 to 56 columns. Of the 1,034 gold queries, 39.5% need a join, 26.8% a `GROUP BY`, 8.0% a nested query and 7.7% a set operation — the constructs where a query can run without error and still return the wrong rows. By the official difficulty classifier, 248 questions are easy, 446 medium, 174 hard and 166 extra hard.

3.2 Architecture

Figure 3.1 shows the system. The reproduced CodeS components build the database prompt, generate four candidate queries and select one. The enhancement executes all four candidates, measures how many return the same result and decides whether to answer or to ask the user. The application wraps this pipeline in a desktop interface, and the evaluation and the application run the same code.

3.3 The reproduced CodeS pipeline

CodeS does not see the raw database. For each question it builds a textual prompt, exactly as in the authors' code (Figure 3.2):

- **Schema sequence:** every table with its columns, types, primary keys, foreign keys and a few representative values per column.
- **Column descriptions:** human-readable names from the dataset's schema file, where they differ from the column name.

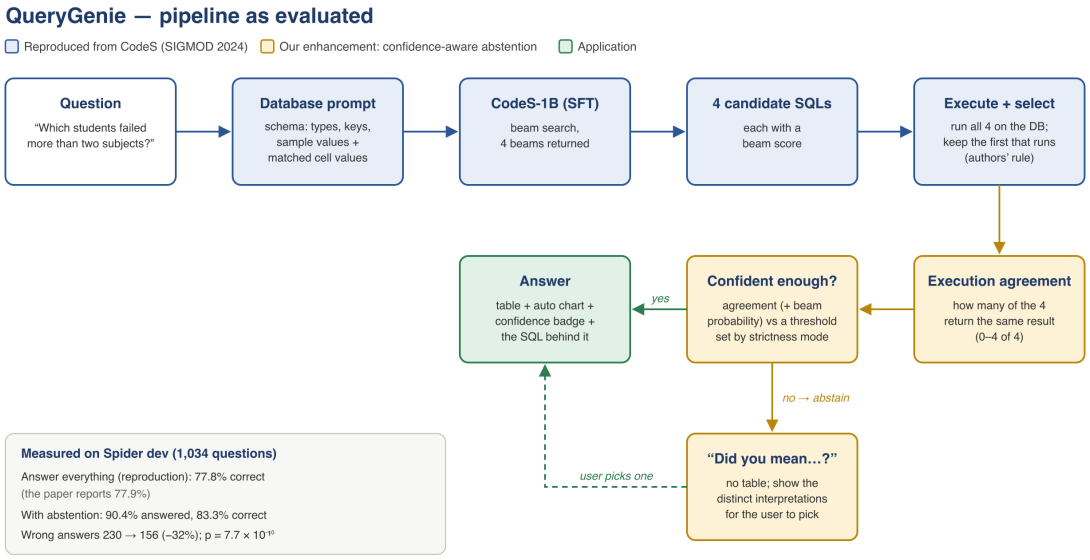


Figure 3.1: System architecture: reproduced CodeS components (blue), the enhancement (amber) and the application (green).

- **Matched contents:** stored values that match words in the question, found by the authors' value retriever (string similarity of at least 0.9, at most 25 values per column).

The question follows, and the model generates four candidates with beam search. The first candidate that SQLite accepts (`EXPLAIN QUERY PLAN`) is returned, otherwise the first beam — the rule in the authors' inference code. Prompts longer than 3,840 tokens are cut from the start, as the authors do; no Spider prompt is that long.

Environment and harness. The authors used eight NVIDIA A800 GPUs; we used a Tesla T4 (Google Colab, fp16, batch size 1, as in their inference) and an Apple M4 for the realistic evaluations, with newer PyTorch and transformers versions. The reproduction passed the department's Week-4 gate (run T1-001). After early runs scored 64.8–73.8%, the harness was audited line by line against the authors' code, which found a value-retriever bug (numbers were never matched, changing the prompt for 12.4% of questions), a simplified scoring metric, missing column

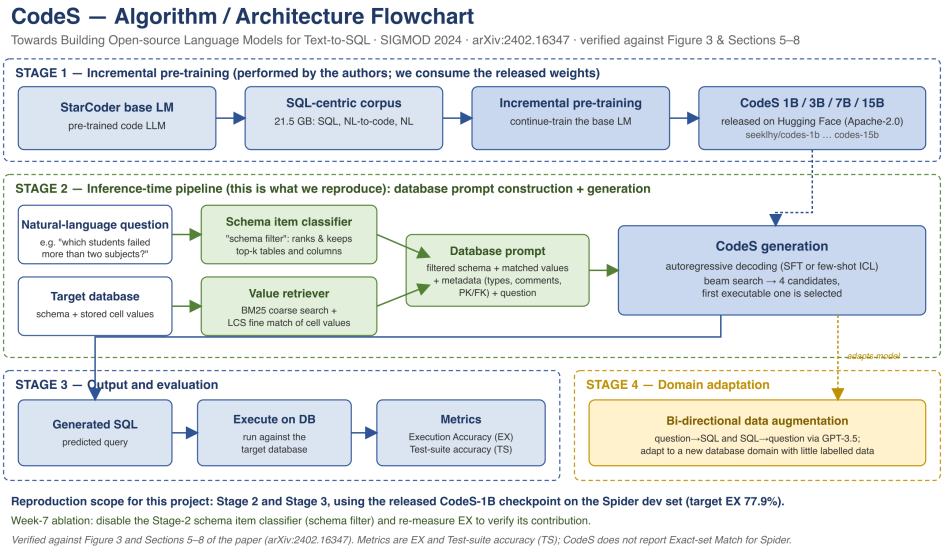


Figure 3.2: The CodeS pipeline, annotated from the paper.

descriptions, and predictions that were not saved. All four were fixed; every run since is scored with the official evaluator [13] and logged with seed, hardware and runtime, including failed and superseded runs.

Deviations. Values are matched by scanning all distinct values instead of the authors' Lucene BM25 pre-filter (no Java dependency; on BIRD a Python word index plays the pre-filter's role); the optional schema classifier is not applied (its checkpoints were unavailable; Spider schemas are small); and test-suite accuracy is TBD, because the distilled test-suite databases were not obtained.

3.4 Proposed approach: confidence-aware abstention

The Week-1 abstract named execution-guided self-correction (DIN-SQL [19]) as the enhancement. The reproduction showed that 97.3% of CodeS's queries already run, so repairing failures could recover at most 2.7 points, while the dominant error is a query that runs but returns the wrong rows. The enhancement therefore became

Algorithm 1: Confidence-aware abstention on top of CodeS

Input: question Q, database D, threshold τ
Output: an answer, or the readings for the user to choose from
$P \leftarrow$ CodeS database prompt for (Q, D);
$(s_1, \ldots, s_4), (b_1, \ldots, b_4) \leftarrow$ four beam candidates and their scores;
for $i \leftarrow 1$ **to** 4 **do**
  $r_i \leftarrow$ result of executing s_i on D (read-only; empty if it fails);
end
$k \leftarrow$ first i whose query passes `EXPLAIN QUERY PLAN`, else 1;
$a \leftarrow \frac{1}{4} |\{\, i : r_i = r_k \,\}|$ if s_k runs, else 0;
$c \leftarrow a + 0.01\, e^{b_k} / \sum_j e^{b_j}$;
if $c \geq \tau$ **then**
  return result r_k with agreement a;
else
  return the distinct results among $r_1, \ldots, r_4$ as readings;
end

confidence-aware abstention, combining self-consistency [15] with selective classification [16]; self-correction was kept as an application feature and measured separately (Chapter 4).

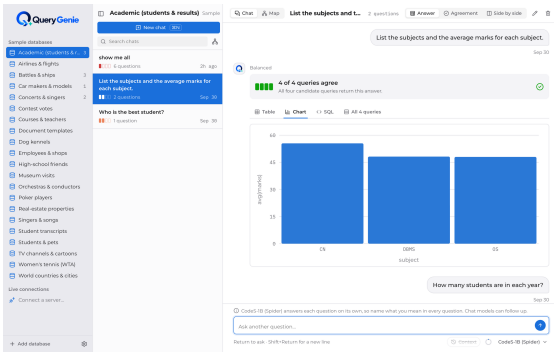
All four beam candidates are executed. The confidence of the returned answer is the share of candidates whose result equals it — execution agreement, from 0 to 4 of 4, and zero when the returned query does not run — with the beam probability used only to break ties. Below a threshold τ the system abstains: it shows no table and offers the distinct results of the candidates as readings for the user to choose from (Algorithm 1). No retraining is involved.

Hypothesis and protocol (fixed before the results were seen). H1: at the same coverage, accuracy on answered questions is higher with agreement-based abstention than with random abstention and than the 77.8% baseline; H0: no difference. Over 10 random 50/50 splits of the development set, the threshold is set on the calibration half for at least 90% coverage and applied to the test half; the paired gain is tested with a one-sided paired t-test and a Wilcoxon signed-rank test ($\alpha = 0.05$), and checked with a 5,000-sample bootstrap over questions. The application uses $\tau =$

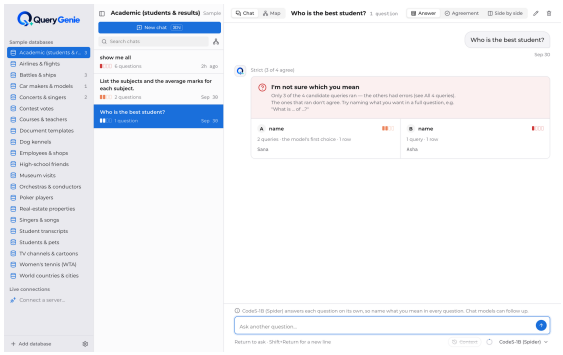
0.2526, the median of the calibration thresholds: it abstains when no candidate runs, or when only 1 of 4 supports the answer and its beam probability is low.

3.5 The application

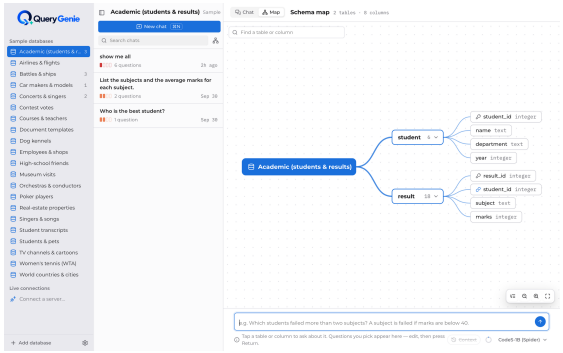
QueryGenie is a desktop application for macOS and Windows. A setup assistant downloads the model (about 4.5 GB) on first start; everything then runs locally on open weights. Questions are asked in chats; every answer shows how many of the four queries agree, the table, an automatic chart, the SQL and all four candidates, and when the candidates disagree it asks which reading was meant (Figure 3.3). A schema map draws the database, and users can open their own SQLite files or connect read-only to PostgreSQL and MySQL. Failed queries are sent back to the model once with the database error (self-correction), and an empty answer that misses a value stored in the database is sent back with that value. Other models (Ollama, cloud services) can be chosen; they get the same agreement check, labelled as not calibrated.



(a) An answer: all four queries agree; table, chart and SQL are one click away.



(b) Abstention: the queries disagree, so it asks which reading was meant.



(c) The schema map: tap a table or column to ask about it.

Figure 3.3: QueryGenie, the desktop application.

Chapter 4

Experimental Results

Metric. Execution Accuracy (EX) counts a predicted query as correct when its result equals the gold query's, scored with the official Spider evaluator [13] (and, on BIRD, with BIRD's set comparison of results). For abstention, *coverage* is the share of questions answered and *accuracy on answered* the EX over those. Every number below comes from a logged run; unknown values are marked TBD.

4.1 Reproduction

Table 4.1 lists the runs on the Spider development set. Each step isolates a real effect: the full CodeS prompt raised the early simple-metric score from 64.8% to 71.7%, the authors' selection rule to 73.8%, and the audit showed that the remaining gap was our scoring rather than the model.

Scored with the paper's own evaluator, the reproduced CodeS-1B reaches **77.8% against the reported 77.9%**, a difference of 0.1 points, well inside the department's tolerance. Beam search is deterministic: re-runs returned identical predictions. Accuracy falls with difficulty, from 90.3% on easy to 83.6% on medium, 64.9% on hard and 56.6% on extra-hard questions; hard and extra-hard questions

Table 4.1: Runs on the Spider development set (1,034 questions). "Simple" is the early metric; only "official" matches the paper.

Run	Configuration	Metric	EX
T2-001	schema and foreign keys only; first beam	simple	64.8%
T2-002	full CodeS prompt; first beam	simple	71.7%
T2-003	+ authors' candidate selection (repeated identically)	simple	73.8%
T2-005	audited harness	official	77.8%
T2-006	fresh re-run of T2-005	official	77.76% (804)
Paper	CodeS-1B, Table 5 of [1]	official	77.9%

Table 4.2: Ablation study, official evaluator (runs AB-103 to AB-106).

Component removed	EX	Change	Queries that run
— (full pipeline)	77.8%	—	97.3%
Foreign keys	76.9%	−0.9	95.9%
Representative column values	76.4%	−1.4	98.1%
Candidate selection (first beam only)	76.0%	−1.8	94.1%
Value retriever	75.9%	−1.9	97.2%

are a third of the set but 133 of the 230 wrong answers. Test-suite accuracy (paper: 72.2%) is TBD.

4.2 Ablation study

Each component was disabled in turn and the full development set re-evaluated with the official evaluator (Table 4.2). Every component contributes. Without the value retriever almost as many queries still run, but with the wrong literal values, so the answers are silently wrong; candidate selection is the component that most improves executability, because it replaces a broken first beam with a working one.

4.3 Enhancement: confidence-aware abstention

Table 4.3 compares the original and the enhanced system on the same 1,034 questions and the same predictions (run T2-006), and Table 4.4 gives the significance tests.

Table 4.3: Original against enhanced on the same 1,034 questions.

System	Answered	Accuracy on answered	Wrong answers
Original (reproduced CodeS-1B)	100%	77.76%	230
Enhanced, tested setting	90.4%	**83.32%**	**156**
Enhanced, 3 of 4 must agree	62.8%	86.13%	90
Enhanced, all 4 must agree	39.8%	88.35%	48

Table 4.4: Significance over 10 calibration/test splits and a 5,000-sample bootstrap over questions.

Signal	Mean gain	Paired t-test (df 9)	Wilcoxon	Bootstrap 95% CI
Execution agreement	+5.46 pts	$t = 24.44$, $p = 7.7 \times 10^{-10}$	$W = 55$, $p = 9.8 \times 10^{-4}$	[+4.51, +6.71]
Beam probability (comparison)	+3.08 pts	$t = 10.81$, $p = 9.3 \times 10^{-7}$	$W = 55$, $p = 9.8 \times 10^{-4}$	[+1.97, +3.94]

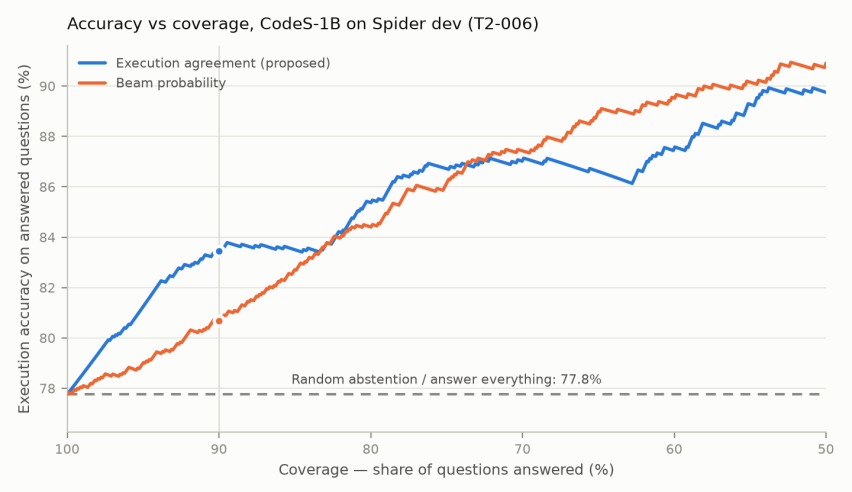


Figure 4.1: Accuracy on answered questions against coverage; the dashed line is the original system (and random abstention).

H0 is rejected: all 10 splits show a gain, and no bootstrap resample shows a gain of zero or less. At the tested setting the system declines 99 questions, 74 of which it would have answered wrongly: wrong answers fall by 32% at the cost of withholding 3.1% of the correct ones. The signal is calibrated — accuracy rises with agreement, from 0% when the selected query does not run (28 questions) to 56.8% at 1 of 4, 75.6% at 2, 82.3% at 3 and 88.3% at 4 of 4 — and Figure 4.1 shows the whole accuracy–coverage curve.

What the enhancement does not do. Plain Execution Accuracy stays at 77.8%:

Table 4.5: The same 508 questions in their original and their realistic wording (run RL-001).

	Original wording	**Realistic wording**
Execution Accuracy, answering everything	72.8% (370)	69.5% (353)
Paper, SFT CodeS-1B (Table 7 of [1])	—	70.9%
Balanced setting: questions answered	88.6%	86.2%
Balanced setting: accuracy on answered	79.3%	76.7%
Wrong answers: answering everything → balanced	138 → 93	155 → 102

the gain is in the answers the system gives, not in answering more questions correctly. Using agreement to *choose* the answer (majority vote) instead of only to decide whether to trust it lowers EX to 75.73%, although a correct query is among the four candidates for 83.08% of questions — room for a learned reranker.

4.4 Self-correction (application feature)

Keeping the T2-006 candidates fixed, every failed candidate (377 queries in 211 questions) was sent back to CodeS-1B once with its database error (run RP-001). Only 61 repaired queries ran and one answer became correct: EX moved from 77.76% to 77.85%, and the tested abstention setting from 83.32% to 83.23% at 90.5% coverage. CodeS was fine-tuned on question–SQL pairs and never learned to use error feedback, which confirms the choice of enhancement. The application's later repairs (stored-value hints for empty answers, joins for misplaced columns) are not measured.

4.5 Benchmark-to-reality gap

The evaluated CodeS-1B pipeline was run unchanged, with the abstention threshold calibrated on Spider. On Spider-Realistic every question is paired with its original Spider wording and both were run on the same machine (run RL-001, Apple M4), so the difference measures the wording alone (Table 4.5).

A second reproduction point. On Spider-Realistic we obtain 69.5% against the paper's 70.9% for SFT CodeS-1B, a difference of 1.4 points, inside the tolerance. As a hardware check, on the 508 original-wording questions this run chose the same query as the Colab run T2-006 for 485 questions, with 72.8% against 73.2% accuracy.

The wording gap. Removing the column names costs 3.3 points (bootstrap 95% CI $[0.8, 5.9]$): 31 questions are right only in the original wording and 14 only in the realistic one (exact McNemar $p = 0.016$), so the drop is real but small. Abstention keeps working on the realistic wording without recalibration: accuracy still rises with agreement (47.1% at 1 of 4, 86.9% at 4 of 4), and the balanced setting removes a third of the wrong answers (155 to 102) at 86.2% coverage.

Real-world databases. The BIRD Mini-Dev evaluation (run BD-001) was in progress when this report was printed; its result is TBD.

Chapter 5

Limitations and Future Work

5.1 Limitations

- **Abstention does not raise overall accuracy.** The system is right more often when it answers, by declining about 10% of questions; Execution Accuracy over all questions stays at the reproduced 77.8%.
- **Agreement is not proof.** 11.7% of the answers on which all four candidates agree are still wrong, and below about 74% coverage the beam probability is the better abstention signal.
- **One model, one calibration.** The threshold was calibrated on Spider with CodeS-1B. Other models in the application get the same agreement check, but their accuracy was not measured.
- **Optimistic split statistics.** The 10 calibration/test splits reuse the same questions, so their p-values are optimistic; the question-level bootstrap is the check that does not depend on this.
- **Single questions.** CodeS was trained on single questions, so with CodeS the application answers each question on its own; follow-up questions need a chat

model, whose accuracy is not measured.

- **Reproduction scope.** Only inference was reproduced, on smaller GPUs and newer libraries than the authors', without their schema classifier and Lucene index; test-suite accuracy is TBD.

5.2 Conclusion

CodeS-1B was reproduced to within 0.1 points of its published Spider result, on hardware far smaller than the authors', after an audit that traced most of an apparent 4–13 point gap to our own evaluation rather than to the model. On top of it, confidence-aware abstention uses the agreement between the four candidates CodeS already produces: at the tested setting it answers 90.4% of questions and is correct on 83.3% of them, against 77.8%, removing a third of the wrong answers, with a significant and calibrated gain. QueryGenie delivers this as a local desktop application that shows how sure it is and asks when it is not.

5.3 Future work

Research.

1. **Test-suite accuracy** from the saved predictions, to report the paper's stricter metric (72.2%).
2. **Larger models**: evaluate CodeS-3B (paper: 83.4%) and calibrate its own threshold.
3. **A learned reranker** to close part of the gap between the 77.8% selected and the 83.1% of questions with a correct candidate, and a combined agreement–probability signal.
4. **Calibration beyond Spider**: tune the threshold on realistic and real-world

data, and measure the other models the application offers.

From application to platform. The longer-term plan is a system that organisations plug into their own databases, keeping the local, open-weight design so that no schema or data leaves the organisation:

5. **Any database**: connectors for SQL Server, Snowflake and Apache Spark alongside the existing SQLite, PostgreSQL and MySQL, with the model's SQL translated to each dialect and checked by the repair loop; document stores such as MongoDB through a SQL gateway that exposes collections as tables.

6. **Per-database adaptation**: a value index built once per database (the authors' BM25 approach) for large tables, and optional one-time fine-tuning on synthetic question–SQL pairs for a specific database, as in the bi-directional data augmentation of the CodeS paper.

7. **Access control**: users and roles that limit which databases, tables and columns each person may query, with an audit log of every question and query.

8. **Serving at scale**: weight quantisation to run larger CodeS models on ordinary laptops, and a shared server for many concurrent users.

References

[1] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "CodeS: Towards Building Open-source Language Models for Text-to-SQL," *arXiv preprint arXiv:2402.16347*, 2024, sIGMOD 2024. [Online]. Available: https://arxiv.org/abs/2402.16347

[2] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," *arXiv preprint arXiv:1809.08887*, 2018, eMNLP 2018. [Online]. Available: https://arxiv.org/abs/1809.08887

[3] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL," *arXiv preprint arXiv:2406.08426*, 2024, tKDE 2025. [Online]. Available: https://arxiv.org/abs/2406.08426

[4] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, and Y. Luo, "A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going?" *arXiv preprint arXiv:2408.05109*, 2024, tKDE 2025. [Online]. Available: https://arxiv.org/abs/2408.05109

[5] A. Mohammadjafari, A. S. Maida, and R. Gottumukkala, "From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems," *arXiv preprint arXiv:2410.01066*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2410.01066

[6] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, "A Survey on Employing Large Language Models for Text-to-SQL Tasks," *arXiv preprint arXiv:2407.15186*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2407.15186

[7] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si, F. Huang, and Y. Li, "A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions," *arXiv preprint arXiv:2208.13629*, 2022, tKDE 2022. [Online]. Available: https://arxiv.org/abs/2208.13629

[8] N. Deng, Y. Chen, and Y. Zhang, "Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect," *arXiv preprint arXiv:2208.10099*, 2022, cOLING 2022. [Online]. Available: https://arxiv.org/abs/2208.10099

[9] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning," *arXiv preprint arXiv:1709.00103*, 2017, arXiv 2017. [Online]. Available: https://arxiv.org/abs/1709.00103

[10] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," *arXiv preprint arXiv:1911.04942*, 2019, aCL 2020. [Online]. Available: https://arxiv.org/abs/1911.04942

[11] X. V. Lin, R. Socher, and C. Xiong, "Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing," *arXiv preprint arXiv:2012.12627*, 2020, eMNLP 2020. [Online]. Available: https://arxiv.org/abs/2012.12627

[12] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-Grounded Pretraining for Text-to-SQL," *arXiv preprint arXiv:2010.12773*, 2020, nAACL 2021. [Online]. Available: https://arxiv.org/abs/2010.12773

[13] R. Zhong, T. Yu, and D. Klein, "Semantic Evaluation for Text-to-SQL with Distilled Test Suites," *arXiv preprint arXiv:2010.02840*, 2020, eMNLP 2020. [Online]. Available: https://arxiv.org/abs/2010.02840

[14] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," *arXiv preprint arXiv:2109.05093*, 2021, eMNLP 2021. [Online]. Available: https://arxiv.org/abs/2109.05093

[15] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," *arXiv preprint arXiv:2203.11171*, 2022, iCLR 2023. [Online]. Available: https://arxiv.org/abs/2203.11171

[16] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," *arXiv preprint arXiv:1705.08500*, 2017, neurIPS 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[17] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL," *arXiv preprint arXiv:2302.05965*, 2023, aAAI 2023. [Online]. Available: https://arxiv.org/abs/2302.05965

[18] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," *arXiv preprint arXiv:2305.03111*, 2023, neurIPS 2023. [Online]. Available: https://arxiv.org/abs/2305.03111

[19] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," *arXiv preprint arXiv:2304.11015*, 2023, neurIPS 2023. [Online]. Available: https://arxiv.org/abs/2304.11015

[20] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," *arXiv preprint arXiv:2308.15363*, 2023, vLDB 2024. [Online]. Available: https://arxiv.org/abs/2308.15363

[21] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, "MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL," *arXiv preprint arXiv:2312.11242*, 2023, aCL 2025. [Online]. Available: https://arxiv.org/abs/2312.11242

[22] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *arXiv preprint arXiv:2405.16755*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2405.16755

[23] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards Robustness of Text-to-SQL Models against Synonym Substitution," *arXiv preprint arXiv:2106.01065*, 2021, aCL 2021. [Online]. Available: https://arxiv.org/abs/2106.01065

[24] Y. Gan, X. Chen, and M. Purver, "Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization," *arXiv preprint arXiv:2109.05157*, 2021, eMNLP 2021. [Online]. Available: https://arxiv.org/abs/2109.05157

[25] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, and K. Yu, "LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations," *arXiv preprint arXiv:2106.01093*, 2021, aCL 2021. [Online]. Available: https://arxiv.org/abs/2106.01093

[26] P. Yin, G. Neubig, W. tau Yih, and S. Riedel, "TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data," *arXiv preprint arXiv:2005.08314*, 2020, aCL 2020. [Online]. Available: https://arxiv.org/abs/2005.08314

[27] T. Yu, C.-S. Wu, X. V. Lin, B. Wang, Y. C. Tan, X. Yang, D. Radev, R. Socher, and C. Xiong, "GraPPa: Grammar-Augmented Pre-Training for Table Semantic Parsing," *arXiv preprint arXiv:2009.13845*, 2020, iCLR 2021. [Online]. Available: https://arxiv.org/abs/2009.13845

[28] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of ChatGPT's zero-shot Text-to-SQL capability," *arXiv preprint arXiv:2303.13547*, 2023, arXiv 2023. [Online]. Available: https://arxiv.org/abs/2303.13547

[29] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. O. Arik, "CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL," *arXiv preprint arXiv:2410.01943*, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2410.01943

[30] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii *et al.*, "StarCoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023, arXiv 2023. [Online]. Available: https://arxiv.org/abs/2305.06161